Informe técnico de revisión
Responsabilidades y dependencias del módulo Media

ALL-133 – Sprint final S1

10 de noviembre de 2026

Resumen

Este documento registra la revisión solicitada para la semana 1 del sprint final. Se analizaron handlers, validación, procesamiento, subida a Cloudinary, persistencia de referencias, manejo de errores y composición del microservicio Media. La línea base es `develop` en `b3dc9ad51c97bccc5bdb8aba8fc44d0449ff7bad`. No se modificó el código Go ni se aplicaron refactors.

1. Resumen ejecutivo

El módulo tiene dos flujos:

1. **Subida múltiple**: `POST/upload/multiple`. Acepta el campo multipart `files`, valida y procesa hasta diez archivos, los sube a Cloudinary y devuelve referencias. No crea filas en BD.
2. **Subida autenticada**: `POST/v1/media/upload`. Exige `HTTPAuth`, acepta `tipo` y exactamente un `file`, utiliza las políticas `PERFIL`/`PORTADA`, sube a Cloudinary y persiste `models.Media`.

La separación de capas es razonable: HTTP está en `internal/api/media`, el pipeline en `internal/service/media` y la composición en `cmd/media/main.go`. Los principales candidatos de refactor son la duplicación de parseo multipart, la ubicación compartida del mapeo de errores, los campos no consumidos de `ProcessedFile` y los mapas paralelos de política por tipo. Los archivos huérfanos en Cloudinary y el rollback de lotes son defectos pendientes, no cambios de esta semana.

2. Estado de la revisión

Elemento	Evidencia
Rama de trabajo	`ALL-133-analisis-tecnico-media`, creada para esta tarea desde `develop`.
Línea base	`b3dc9ad51c97bccc5bdb8aba8fc44d0449ff7bad`.
Estado inicial	Rama `develop` limpia y alineada con `origin/develop`; no había cambios locales.
Rutas revisadas	`backend-go/internal/api/media`, `backend-go/internal/service/media`, `backend-go/cmd/media/main.go`; consumidores buscados en todo `backend-go`.
Validación ejecutada	Desde `backend-go`: `gotest./internal/api/media./internal/service/media`. Resultado: ambos paquetes `ok`.
Cambios de aplicación	Ninguno. Solo se crea este informe.

3. Arquitectura y dependencias

```
main
  -> UploadService(CloudinaryUploader, FileValidator, ImageProcessor)
       -> UploadHandler -> /upload/multiple
  -> MediaService(UploadService, MediaRepository)
       -> MediaHandler + HTTPAuth -> /v1/media/upload
```

3.1. Responsabilidades por archivo

Las rutas api/ y service/ de la tabla se resuelven bajo `backend-go/internal/`; cmd/ bajo `backend-go/`. Los handlers citados después pertenecen al paquete API; los demás archivos de servicio a `backend-go/internal/service/media/`. Las líneas son las de b3dc9ad.

Ruta y líneas	Responsabilidad, entrada, salida y efectos
`api/media/upload_handler.go:23--60`	Recibe POST multipart, limita y parsea body, obtiene `files`, llama a `ProcessFiles` y devuelve JSON 200 o error. Puede producir assets Cloudinary, no BD.
`api/media/media_handler.go:32--78`	Exige ID de usuario en contexto, normaliza `tipo`, exige un `file`, llama a `MediaService.Upload` y devuelve JSON 201 con id, tipo, URL y PublicID.
`service/media/upload_service.go:79--112`	Orquesta lectura, validación, procesamiento y subida a Cloudinary. Devuelve `models.UploadResult`.
`service/media/upload_service.go:118--134`	Abre cada header y lee todo a memoria para producir `[]File`.
`service/media/validator.go:73--126`	Comprueba vacío, tamaño, cantidad y MIME real por magic bytes; el lote falla completo ante un archivo inválido.
`service/media/processor.go:65--148`	Decodifica, redimensiona y recomprime JPEG/PNG/WEBP; AVIF pasa sin transcodificar.
`service/media/media_service.go:59--89`	Selecciona carpeta/dimensión por tipo, ejecuta el upload y crea `models.Media`.
`service/media/media_service.go:32--45`	Adapta GORM mediante `MediaRepository.Create`; inserta la referencia en BD.
`cmd/media/main.go:25--52`	Conecta BD, inicializa dependencias y registra los dos endpoints; el segundo queda detrás de `HTTPAuth`.

4. Flujos y efectos persistentes

4.1. Subida múltiple

La secuencia es: parseo multipart → lectura a memoria → validación completa → procesamiento completo → subida individual a Cloudinary → respuesta. La validación y el procesamiento son todo-o-nada; la subida externa no es atómica. Si una subida posterior falla, las anteriores pueden quedar creadas.

4.2. Subida autenticada

La secuencia es: autenticación → parseo → tipo y cardinalidad → selección de política → validación/procesamiento → Cloudinary → GORM. Si GORM falla después de Cloudinary, el asset remoto puede quedar sin fila correspondiente.

5. Comportamientos a preservar

- Solo POST; otros métodos devuelven 405.
- Campos multipart: `files` para múltiple y `file` para autenticado.
- Ausencia de usuario en contexto: 401.
- Tipos válidos: `PERFIL` y `PORTADA`; otro tipo: 400.
- Límite por defecto: 5 MiB por archivo; máximo múltiple: 10 archivos.
- Detección por magic bytes, no por extensión ni Content-Type del cliente.
- Validación y procesamiento de lote antes de comenzar las subidas.
- JPEG, PNG y WEBP se transcodifican; AVIF pasa sin tocar.
- El endpoint autenticado conserva 201 y JSON `id`, `tipo`, `url`, `publicId`.
- BD no disponible: 503 en subida autenticada; subida múltiple independiente de la BD.

6. Hallazgos y cambios propuestos

6.1. R1 – Duplicación de parseo multipart (fuera de S1)

Archivos/líneas: `upload_handler.go:11--48` y `media_handler.go:12--59`.

Ambos handlers repiten constantes, `MaxBytesReader`, `ParseMultipartForm` y la traducción del error de límite. La diferencia válida es el límite efectivo. La duplicación puede causar respuestas distintas al corregir un handler. Propuesta futura, expresamente fuera de S1 porque crea una función nueva. No se implementó. Ejemplo conceptual:

```
func parseMultipart(w http.ResponseWriter, r *http.Request,
    limit int64) (*multipart.Form, error) {
    r.Body = http.MaxBytesReader(w, r.Body, limit)
    if err := r.ParseMultipartForm(multipartMemoryBuffer); err != nil {
        return nil, err
    }
    return r.MultipartForm, nil
}
```

Debe comprobarse que 413 y 400, mensajes y límites permanezcan iguales.

6.2. R2 – Ubicación ambigua de mapUploadError

Archivo/líneas: `upload_handler.go:62--97`; usado también desde `media_handler.go:64--73`.

La función está en el archivo del handler múltiple, pero es una traducción compartida por ambos handlers. Funciona por pertenecer al mismo paquete, aunque la ubicación induce a pensar que es exclusiva del endpoint múltiple. Se propone moverla a `internal/api/media/errors.go`, conservando firma, status y textos:

```
// errors.go, paquete api/media
func mapUploadError(err error) (status int, message string) {
    // misma tabla de errores; solo cambia la ubicación
}
```

6.3. R3 – Datos de ProcessedFile sin consumidor productivo

Archivo/líneas: `processor.go:22--30,65--101`; consumidor `upload_service.go:96--108`.

La búsqueda global muestra que producción usa solo `Name` y `Data`. `Format` y `WasTranscoded` se producen y se comprueban en tests, pero no son usados por el siguiente bloque ni por otro consumidor productivo. Transportarlos agrega superficie interna. Cambio condicionado a confirmar que no se requieren para observabilidad:

```
type ProcessedFile struct {
    Name string
    Data []byte
}
```

Después deben ajustarse tests para verificar bytes, dimensiones, passthrough AVIF y nombre. No debe eliminarse `ValidatedFile.Format`, porque sí se usa para decidir la transcodificación.

6.4. R4 – Dos mapas paralelos por tipo

Archivo/líneas: `media_service.go:20--30,64--69`.

`mediaFolders` y `mediaMaxDimension` usan las mismas claves. Olvidar una entrada puede producir una política incompleta. Se propone consolidar ambos valores:

```
type mediaPolicy struct {
    folder string
    maxDimension int
}
var mediaPolicies = map[string]mediaPolicy{
    models.TipoMediaPerfil:  {folder: "media/perfil", maxDimension: 512},
    models.TipoMediaPortada: {folder: "media/portada", maxDimension:
        1600},
}
```

La política nueva debe conservar tipos, carpetas y dimensiones actuales.

6.5. R5 – Responsabilidad amplia de UploadService

Archivo/líneas: `upload_service.go:21--134`.

El servicio lee multipart, valida, procesa, sube y construye resultados. Es una secuencia coherente, pero concentra responsabilidades. No se recomienda dividirlo en esta semana porque aumentaría dependencias y riesgo. Si se extraen componentes después, `ProcessFiles` debe mantenerse como fachada durante la migración.

7. Defectos e integraciones para semanas posteriores

7.1. P1 – Asset huérfano al fallar persistencia

Archivo/líneas: `media_service.go:78--87`. Cloudinary se actualiza antes de GORM. Si `repo.Create` falla, se devuelve `ErrMediaPersist` pero no se elimina el asset. Debe resolverse con compensación, cola de limpieza o reconciliación; no existe una transacción distribuida real.

7.2. P2 – Lote parcialmente subido

Archivo/líneas: `upload_service.go:95--111`. Si falla el archivo n, los anteriores permanecen en Cloudinary aunque la operación devuelva error. Se debe decidir rollback compensatorio, resultado parcial o proceso asíncrono; cambiarlo afecta comportamiento y operación.

7.3. P3 – Lectura completa antes de validar tamaño

Archivo/líneas: `upload_service.go:118--132` y `validator.go:73--82`. `io.ReadAll` ocurre antes de la validación por archivo. El endpoint HTTP limita el body, pero una llamada directa puede consumir más memoria. Posteriormente debe diseñarse lectura acotada sin confiar únicamente en el tamaño declarado.

7.4. P4 – Magic bytes no garantizan decodificación

Archivo/líneas: `validator.go:84--104`; test `validator_test.go:235--243`. Un JPEG corrupto puede pasar el detector y luego ser rechazado por `image.Decode` en `processor.go:75--83`. Es una limitación documentada, no un refactor de esta semana.

7.5. P5 – Pérdida de transparencia PNG

Archivo: `processor.go:87--100`; test `TestLimitacionConocida_PNGConTransparenciaPierdeElAlfa`. Transcodificar todo a JPEG elimina alfa. Si se agregan logos transparentes, se necesitará otra política de formato.

7.6. P6 – Cobertura insuficiente de persistencia

Hay integración real para Cloudinary, pero no se encontró una prueba equivalente de `MediaHandler->MediaService->MediaRepository` que cubra 201, 401, 400, 503 y fallo de persistencia. Es una brecha de evidencia, no una prueba de fallo.

8. Validación antes/después

Área	Evidencia actual	Criterio posterior
Build/tests	`gotest./internal/api/media./internal/service/media`; pasa.	Debe seguir pasando.
HTTP/límites	Tests cubren método, body, tamaño y lotes.	Status, mensajes y límites efectivos iguales.
Formatos	Tests cubren JPEG, PNG, WEBP, AVIF e inválidos.	Igual detección y transcodificación.
Todo-o-nada previo	Tests evitan subidas si falla validación/procesamiento.	Ninguna subida antes de ambas etapas.
Persistencia	Cobertura directa insuficiente.	Agregar pruebas antes de modificar este flujo.
JSON	`mediaResponse` en `media_handler.go:23--28`.	Mismos campos y 201.

```
cd backend-go
gofmt -d internal/api/media internal/service/media cmd/media/main.go
go test ./internal/api/media ./internal/service/media
go test ./...
go test -tags=integration ./internal/service/media
```

La prueba de integración requiere credenciales y conectividad con Cloudinary y no se ejecutó para no afectar recursos externos.

9. Entrega, archivos y conclusión

El informe se entrega en `DocumentacionSprintFinal/analisistecnicomedia/` dentro de la rama `ALL-133-analisis-tecnico-media`. Los archivos revisados fueron los handlers de `internal/api/media`, los servicios y tipos de `internal/service/media`, `cmd/media/main.go` y `internal/models/media.go`. No hay commit de refactor ni PR de código porque no se aplicaron cambios.

La revisión identifica responsabilidades, llamadores, salidas, efectos persistentes, duplicación, datos no consumidos y riesgos. La entrega es una revisión documental lista para evaluación del equipo, no un refactor ejecutado. R2 y R4 son candidatos acotados; R3 exige preservar sus consumidores de prueba. R1 y R5 quedan fuera de S1. El cierre requiere publicar y enlazar PR y commits y obtener aceptación del equipo.

10. Llamadores, consumidores y decisiones verificadas

Bloque	Llamadores, consumidores y dependencias
UploadHandler.Handle:23	main.go:51 registra la ruta; upload_handler_test.go:113 prueba el handler. Su resultado JSON lo recibe el cliente HTTP.
MediaHandler.Handle:32	main.go:52 lo envuelve en HTTPAuth. Cliente explícito: frontend-marketplace/src/app/test-media/page.tsx:32; lee la URL devuelta.
ProcessFiles:79	UploadHandler:52, MediaService:69 y prueba de integración:34. Lee multipart, valida, procesa y usa el SDK de Cloudinary.
readMultipartFiles:118	ProcessFiles:80 es su llamador; ValidateAll consume los File resultantes. Abre y cierra archivos; lee contenido completo a memoria.
ValidateAll:108 / Validate:73	ProcessFiles:85 llama ValidateAll y este llama Validate:119; mimetype.Detect verifica MIME. ProcessAll consume los validados. Sin BD ni red.
ProcessAll:107 / Process:65	ProcessFiles:90 llama ProcessAll y este llama Process:110. Usa image.Decode, resizeIfNeeded:122 y jpeg.Encode; devuelve bytes. Sin persistencia.
WithFolder:61 / WithMaxDimension:69	MediaService:69 configura copias; integración:30 usa WithFolder. WithMaxDimension también copia el procesador. No mutar el servicio compartido original.
MediaService.Upload:59	MediaHandler:63 es su llamador; consume URL/PublicID y entrega modelo al handler. Invoca repo.Create:84 después de subir.
gormMediaRepository.Create:43	Implementa MediaRepository usado por MediaService. GORM WithContext/Create inserta tabla media y completa ID/fecha.
mapUploadError:66	UploadHandler:54 y MediaHandler:71 consumen status/mensaje; writeError:90 delega a writeJSON:94.
FileError.Error:39 / Unwrap:46	Lector, validador y procesador producen FileError; API usa errors.As/Is y Error. Nombre/MIME/tamaño aparecen en el mensaje público.

10.1. Datos internos y consumidores públicos

- ProcessedFile.Format y WasTranscoded no tienen lecturas productivas encontradas, pero sí consumidores en `backend-go/internal/service/media/processor_test.go:76,79,140,143,159,162`. R3 no autoriza borrar aserciones sin reemplazarlas: comprobar formato de bytes, dimensiones y equivalencia AVIF.
- ValidatedFile.Format se usa en processor:66 y 79; conservar. ExtensionMismatch se usa en validator_test:181 y 189, y consume Format.Extension. No se propone su eliminación.
- `backend-go/internal/models/media.go:26--36` define FileInfo (name,size,url,publicId opcional) y UploadResult (success,files). Aunque MediaService solo use URL/PublicID, el handler múltiple publica todos esos campos: no eliminarlos.
- MediaFolders/mediaMaxDimension tienen sus consultas en media_service.go:64 y 69. R4 debe reemplazar ambas consultas, además de las declaraciones, conservando todos los valores y el rechazo de tipos inválidos.

Búsquedas reproducibles desde la raíz:

```
rg -n 'ProcessFiles|WithFolder|WithMaxDimension|ProcessedFile|
   WasTranscoded|ExtensionMismatch|mediaFolders|mediaMaxDimension' backend
   -go --glob '*.go'
rg -n '\.Format|FileInfo|UploadResult|PublicID' backend-go --glob '*.go'
```

El alcance de la búsqueda es el repositorio, no consumidores externos desconocidos.

10.2. Ramas y nombres ambiguos

No se identificó una rama inequívocamente redundante para eliminar en S1. El chequeo de archivos vacíos del handler mantiene un mensaje HTTP propio; el validador protege llamadas directas. La guarda len(res.Files) != 1 en media_service:73 es defensiva y se conserva. ProcessFiles no solo procesa, también sube: documentar su efecto es preferible a renombrar el método en S1.

El comentario en `backend-go/internal/service/media/media_service.go:60` sugiere evitar todos los huérfanos, pero la persistencia puede fallar después. Mejora documental propuesta, no aplicada:

```
// Comprobar repositorio y tipo antes de subir.
// Si la persistencia posterior falla, el asset puede quedar huerfano.
```

`backend-go/internal/service/media/errors.go:10` documenta formato no soportado como 400; mapUploadError devuelve 415 para FileError con esa causa. Corregir comentario, no status:

```
// HTTP: ErrEmptyFile -> 400; ErrUnsupportedFormat -> 415
// cuando estan envueltos en FileError.
```

11. Contrato de errores y persistencia

Condición	Resultado actual
Método distinto de POST	Handler devuelve 405; en ruta autenticada el middleware puede rechazar antes.
Parseo inválido / exceso de body	400 / 413; conservar textos distintos por endpoint.
FileError vacío/lectura; demasiado grande; formato inválido	400; 413; 415. Otros FileError también caen en 400: no reclasificarlos en S1.

ErrNoFiles / ErrTooManyFiles	400. Por archivo se comprueba vacío, tamaño, MIME, en ese orden.
Tipo inválido / repositorio ausente	400 / 503. Se comprueba repositorio primero; si concurren ambos problemas prevalece 503.
Fallo Cloudinary / ErrMediaPersist	500 por fallback, con mensaje actual. Sin compensación.
Éxito múltiple / autenticado	200 con success/files / 201 con id,tipo,url,publicId.

El Size público representa bytes procesados y Name conserva el original aun si PNG se convierte a JPEG. El modelo persistente en `backend-go/internal/models/media.go:13--24` incluye usuario, tipo, URL, PublicID único, ID y fecha automáticos en tabla media. MediaService no actualiza el perfil IAM.

12. P7 – Integración pendiente con Nginx y frontend

Clasificación: integración futura. Evidencia estática del repositorio, no verificación del clúster desplegado.

- `frontend-marketplace/src/app/test-media/page.tsx:5,32`: página temporal llama directamente a localhost:8083. Es un flujo de prueba local.
- `docker-compose.yml:134--157`: expone 8083 y configura BD propia, Cloudinary y JWT. No se ejecutó Compose.
- `backend-go/k8s/gateway/configmap.yaml:44--239`: hay rutas IAM, catálogo, órdenes y chat, pero no /v1/media ni /upload/multiple; fallback 404.
- Ese ConfigMap:26 establece client_max_body_size 10m; UploadHandler:33 permite hasta 50 MiB de cuerpo por defecto. Una nueva ruta requiere conciliar límites de gateway, ingress y servicio.
- `backend-go/internal/middleware/jwt_http.go:17,40,45`: verifica JWT localmente; no envía el archivo a IAM.
- `backend-go/internal/service/auth_service.go:295--298,359--389`: IAM valida y guarda URLs de perfil por separado. El pipeline Media no llama a IAM ni catálogo. Cooperación funcional no equivale a una cadena de llamadas de red.

Flujo comprobado: navegador de prueba a Media; Media a Cloudinary y su BD; URL al navegador. Flujo futuro a evaluar: navegador a gateway a Media. No se concluye que acceder directo sea incorrecto para desarrollo local. Para producción faltan decisiones de despliegue/Service, rutas, TLS, CORS, Authorization, tiempos y límites.

Criterios futuros: subir por dominio real sin localhost; comprobar 201, autenticación, URL/fila y lotes en todos los límites. No copiar proxy_pass sin confirmar upstream. Coordinar Media, infraestructura y frontend. No se cambió infraestructura en S1.

13. P8 – Margen del cuerpo multipart

`backend-go/internal/api/media/upload_handler.go:33--34` limita todo el cuerpo a tamaño por archivo multiplicado por cantidad, sin margen para cabeceras y delimitadores. Diez archivos de exactamente 5 MiB requieren más de 50 MiB con multipart; por inspección se deduce rechazo por límite total. El handler autenticado sí suma margen en media_handler:44. No se reprodujo contra servidor real. Añadir margen modifica entradas aceptadas: corrección futura con prueba del borde y máximo total explícito.

14. Alcance acotado y backlog futuro

La entrega elegida es análisis verificable, sin cambios de aplicación. No se exige inventar un refactor para justificar la semana.

- Candidatos compatibles con S1 si se autoriza implementación: R2 (mover una función existente, no crear otra), R4 (consolidar datos internos), comentarios y nombres locales. R3 condicionado a evidencia y pruebas equivalentes.
- Fuera de S1: R1 (función nueva), R5 (separación más amplia) y cambios de endpoints, negocio, persistencia, infraestructura o comportamiento.
- P1/P2 pueden agruparse como consistencia y limpieza: probar fallos parciales y definir compensación antes de implementarla.
- P3 requiere evaluación de memoria; bytes comprimidos no acotan memoria de imagen decodificada.
- P4 es una limitación: AVIF evita Decode y resize en processor:66–72, por lo que no se garantiza decodificación ni límite de dimensiones para ese formato. Revisar requisitos antes de corregir.
- P5 necesita decisión sobre transparencia; P6 pruebas del flujo autenticado; P7 integración de despliegue; P8 borde multipart.

Planificar por separado significa tratar estas decisiones fuera del refactor de S1, no crear obligatoriamente ocho issues ni ocho informes. No se crearon tickets. Cada tarea aceptada debe tener alcance, responsable y comprobación. La hora de capacidad es estimación del equipo; no se presenta como tiempo efectivamente medido.

15. Evidencia final y transferencia

15.1. Ejecutado y no ejecutado

- Estado inicial limpio, develop en b3dc9ad. La coincidencia con origin/develop corresponde a referencia local; no se hizo fetch para certificar estado remoto actual.
- Rama ALL-133-analisis-tecnico-media; commit documental inicial 2b55e1c; esta revisión se registra en commit posterior.
- `gitdiffb3dc9ad--backend-gofrontend-marketplacedocker-compose.yml`: sin diferencias al preparar entrega.
- Pruebas sin caché: `gotest-count=1./internal/api/media./internal/service/media`: API ok, 2.359 s; servicio ok, 3.400 s. Ejecución inicial: ok, 6.765 s y 9.090 s. No son benchmarks.
- PDF se comprueba con dos pasadas de pdflatex y revisión de errores/desbordamientos; diff con git diff –check.
- No ejecutados: suite global, integración real Cloudinary/BD, despliegue, carga ni pruebas HTTP reales. Los comandos anteriores de suite global e integración son recomendaciones, no evidencia de ejecución.

La comparación antes/después es de revisión documental: código idéntico y pruebas focalizadas exitosas. No se afirma haber probado un refactor inexistente. Para un refactor futuro, ejecutar primero pruebas de caracterización de MediaHandler/MediaService con dobles: usuario ausente, tipo inválido, repo nulo, fallo de inserción, éxito y políticas 512/1600. Esas pruebas nuevas no se implementaron en esta entrega.

15.2. Trazabilidad de aceptación

Criterio	Evidencia y estado
Inventario y rutas	Tablas de responsabilidades/llamadores y hallazgos con líneas base.
Consumidores antes de eliminación	Búsquedas, tests y contratos JSON; ninguna eliminación aplicada.
Preservación antes/después	Matriz de contratos/errores, pruebas focalizadas y plan de regresión.
Sin capacidades nuevas	Diff de aplicación vacío, R1 aplazado explícitamente.
Defectos separados	P1–P8 clasificados; no se confunden con refactor realizado.
PR y commit enlazados	Commit inicial 2b55e1c y commit final en Git. URL de PR pendiente de publicación por el responsable.

Fuente, PDF, README y descripción de PR forman la entrega; auxiliares no se versionan. Otro integrante o IA debe comprobar HEAD, preservar cambios posteriores y localizar por símbolo si cambiaron las líneas. Los fragmentos son propuestas, no parches completos: R2 exige mover el cuerpo original íntegro, no sustituirlo por el comentario del ejemplo.

Publicar/enlazar la PR en ALL-133 y la aprobación del equipo son los pasos administrativos pendientes. El informe no certifica producción ni declara resueltos los defectos.